

LINGUAGEM DE PROGRAMAÇÃO C++

Curso de Capacitação em Desenvolvimento Full Stack - MÓDULO 08 (40H)

Elaborado por: Prof(a). Esp. Érika D. G. R. dos Santos

ÁRVORE

Paixão por Desenvolver Talentos

AULA 9: LINGUAGEM DE PROGRAMAÇÃO C++

Conceito de Árvore

Na Computação, uma árvore é uma estrutura de dados hierárquica — muito parecida com uma árvore de verdade, mas “de cabeça para baixo”:

No topo temos um nó raiz (root);

Abaixo dele, galhos (filhos);

E cada nó pode gerar novos ramos (subárvores).

Essa estrutura é usada para organizar informações de forma hierárquica, como:

pastas e arquivos no computador,

estrutura de menus em um aplicativo,

ou relacionamentos familiares (pais e filhos).

Conceito de Árvore

Na Computação, **uma árvore é uma estrutura de dados hierárquica** — muito parecida com uma árvore de verdade, mas “de cabeça para baixo”:

- No topo temos um **nó raiz (root)**;
- Abaixo dele, **galhos (filhos)**;
- E cada nó pode gerar **novos ramos (subárvores)**.

Essa estrutura é usada para **organizar informações de forma hierárquica**, como:

- pastas e arquivos no computador,
- estrutura de menus em um aplicativo,
- ou relacionamentos familiares (pais e filhos).



Conceito de Árvore

Imagine uma **árvore genealógica de família**.

Nela, cada pessoa é um **nó**.

- O **bisavô** está no topo (raiz).
- Ele tem **filhos**, que têm **netos**, e assim por diante.

Cada nível da família representa **um nível da árvore**.

- Cada pessoa “herda” certas características (como nome da família ou sobrenome) dos pais, **assim como as classes em POO herdam atributos e métodos**.
- Mas ao mesmo tempo, **cada um pode ter suas próprias características**, o que lembra a **especialização** nas subclasses.

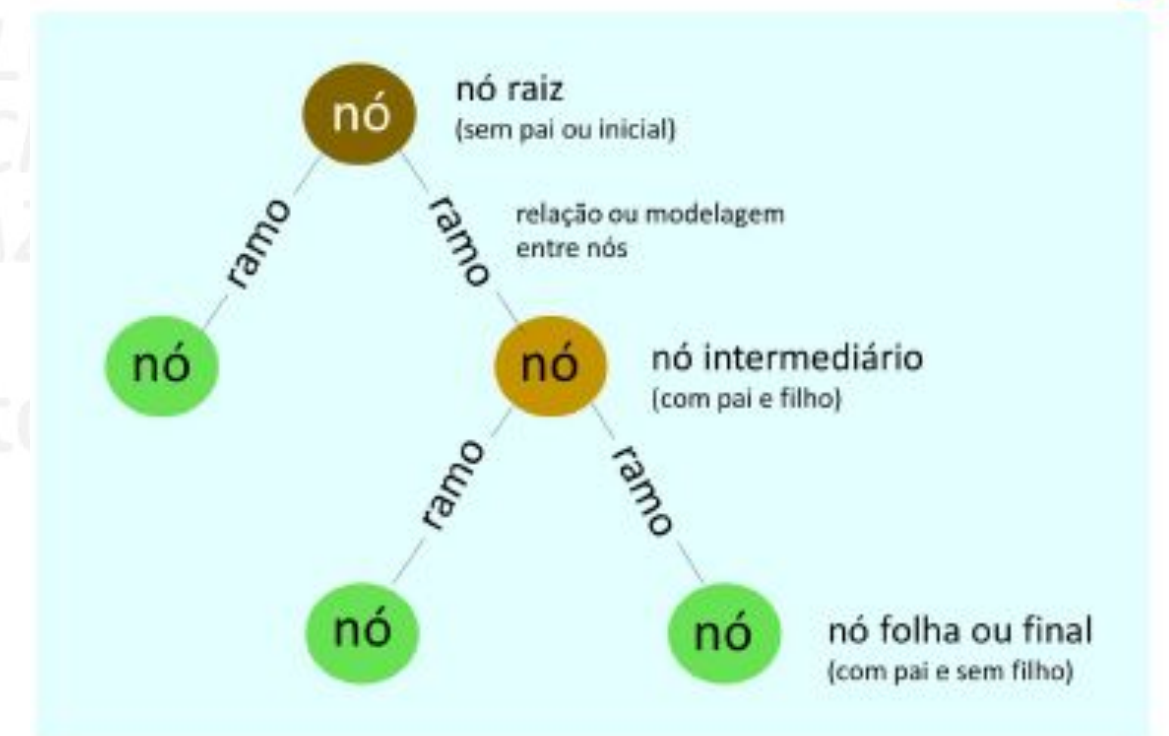


Conceito de Árvore

Na Computação, uma **árvore de dados** funciona exatamente assim:

- existe um **nó principal (raiz)**;
- cada elemento pode **gerar outros (filhos)**;
- e o sistema pode **percorrer a estrutura** para buscar, inserir ou organizar informações.

Essa forma de organizar dados é chamada de **estrutura hierárquica** — muito usada quando há relações de “pai e filho”.

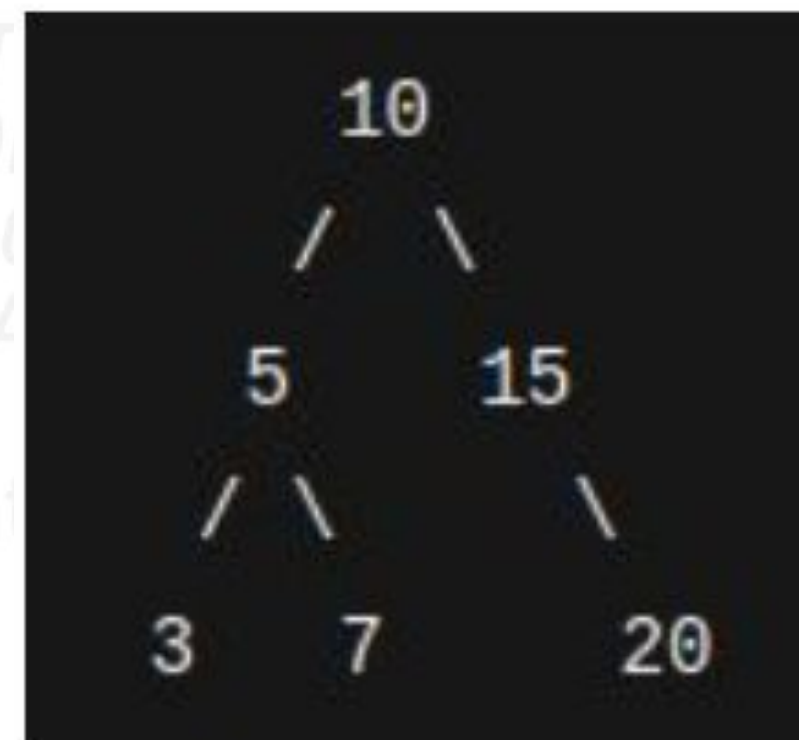


Árvore Binária

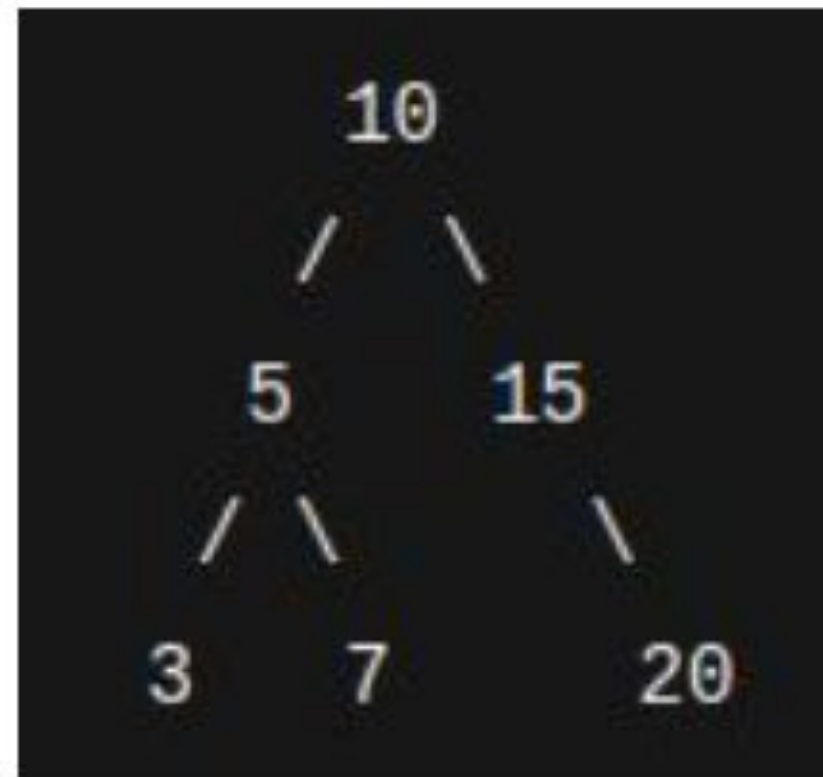
Uma árvore binária é um tipo especial de árvore onde cada nó pode ter, no máximo, dois filhos:

- o filho esquerdo
- e o filho direito

Ela é muito usada em algoritmos e pesquisas (como busca binária ou ordenação), pois permite percorrer dados de forma rápida e organizada.



Árvore Binária



O nó 10 é a raiz; 5 e 15 são filhos; e cada um deles pode ter seus próprios filhos.



Árvore Binária

O nó 0 (valor 10) é a raiz da árvore.

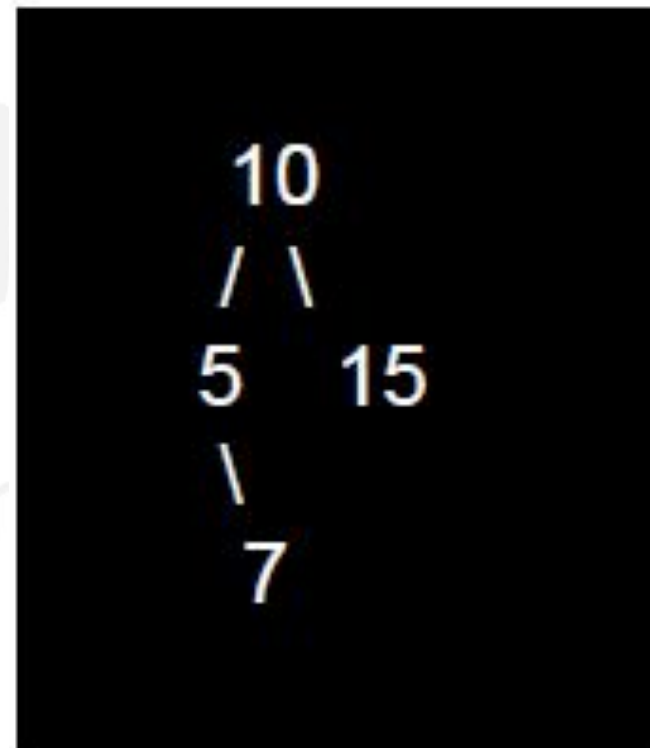
Ele tem dois filhos:

- Esquerda → Nó 1 (valor 5)
- Direita → Nó 2 (valor 15)

O nó 1 (valor 5) tem um filho direito (nó 3, valor 7)

O nó 2 (valor 15) é uma folha (sem filhos)

O nó 3 (valor 7) também é uma folha



Índice	Valor	Esquerda	Direita
0	10	1	2
1	5	None	3
2	15	None	None
3	7	None	None



Pense no **Gerenciador de Arquivos do Windows, Linux ou macOS**:

- A **pasta “Documentos”** é a raiz.
- Dentro dela, há **outras pastas (filhas)**.
- E dentro de cada uma, há **arquivos ou novas subpastas**.

Cada **pasta** ou **arquivo** é um **nó da árvore**.

Os **sistemas operacionais** usam árvores para representar esse tipo de estrutura e permitir:

- navegação (abrir subpastas),
- buscas rápidas,
- organização hierárquica.



Vamos praticar?



Desafio

Na Floresta Encantada, cada personagem tem um nome, um nível de poder (número inteiro) e um tipo (como Animal ou Mago). Os personagens mais poderosos ficam à direita da árvore, e os menos poderosos à esquerda. Alguns personagens são especiais: Chefes da Floresta (herdam de Personagem) e têm um atributo extra chamado reino. A árvore deve permitir percorrer os personagens do menos poderoso para o mais poderoso, exibindo todas as informações.

Criar a árvore da floresta.

Adicionar pelo menos 6 personagens:

- Pelo menos 2 devem ser Chefes.
- Exemplos: Lupi (Animal), Mago Merlin, Chefe Dragão, etc.

Percorrer a árvore em ordem e exibir informações de todos os personagens.



HERANÇA E POLIMORFISMO

AULA 10: LINGUAGEM DE PROGRAMAÇÃO C++

Classes Abstratas

Funções Virtuais Puras

- Classe abstrata: não pode ser instanciada diretamente
- Função virtual pura: `virtual void som() = 0;`
- Força as classes derivadas a implementarem o método
- Exemplo: `Forma` → `Circulo`, `Retangulo` (cada um calcula área de forma diferente)

Pontos importantes sobre Classes Abstratas:

1. **Método virtual puro:** `virtual void funcao() const = 0;`
2. **Não podem ser instanciadas:** `Forma f; // ERRO!`
3. **Devem ser implementadas por subclasses** (ou a subclasse também será abstrata)
4. **Destrutor virtual:** Essencial para limpeza correta
5. **Ponteiros/Referências:** Usar para polimorfismo

```

#include <iostream>
#include <vector>
#include <cmath>
#include <memory>

using namespace std;

// Classe abstrata (interface)
class Forma {
public:
    // Métodos virtuais puros -> tornam a classe abstrata
    virtual double calcular_area() const = 0;
    virtual double calcular_perimetro() const = 0;
    virtual void exibir() const = 0;
    // Destrutor virtual (importante para polimorfismo)
    virtual ~Forma() = default;
};

// Classe concreta: Círculo
class Circulo : public Forma {
private:
    double raio;

public:
    Circulo(double r) : raio(r) {}

    double calcular_area() const override {
        return M_PI * raio * raio;
    }

```

```

    }

    double calcular_perimetro() const override {
        return 2 * M_PI * raio;
    }

    void exibir() const override {
        cout << "Círculo (raio=" << raio << ")" << endl;
        cout << "  Área: " << calcular_area() << endl;
        cout << "  Perímetro: " << calcular_perimetro() << endl;
    }
};

```

```

// Classe concreta: Retângulo
class Retangulo : public Forma {
private:
    double largura;
    double altura;

public:
    Retangulo(double l, double a) : largura(l), altura(a) {}

    double calcular_area() const override {
        return largura * altura;
    }

    double calcular_perimetro() const override {

```



```
double calcular_perimetro() const override {
    return 2 * (largura + altura);
}

void exibir() const override {
    cout << "Retângulo (largura=" << largura << ", altura=" << altura << ")" << endl;
    cout << "  Área: " << calcular_area() << endl;
    cout << "  Perímetro: " << calcular_perimetro() << endl;
}

};

// Classe concreta: Triângulo
class Triangulo : public Forma {
private:
    double lado1, lado2, lado3;

public:
    Triangulo(double l1, double l2, double l3) : lado1(l1), lado2(l2), lado3(l3) {}

    double calcular_area() const override {
        // Fórmula de Herão
        double s = (lado1 + lado2 + lado3) / 2;
        return sqrt(s * (s - lado1) * (s - lado2) * (s - lado3));
    }
}
```



```

double calcular_perimetro() const override {
    return 2 * (largura + altura);
}

void exibir() const override {
    cout << "Retângulo (largura=" << largura << ", altura=" << altura << ")" << endl;
    cout << "  Área: " << calcular_area() << endl;
    cout << "  Perímetro: " << calcular_perimetro() << endl;
}
};

```

// Classe concreta: Triângulo

```

class Triangulo : public Forma {
private:
    double lado1, lado2, lado3;

public:
    Triangulo(double l1, double l2, double l3) : lado1(l1), lado2(l2), lado3(l3) {}

```

```

double calcular_area() const override {
    // Fórmula de Herão
    double s = (lado1 + lado2 + lado3) / 2;
    return sqrt(s * (s - lado1) * (s - lado2) * (s - lado3));
}

```

```

double calcular_perimetro() const override {
    return lado1 + lado2 + lado3;
}

void exibir() const override {
    cout << "Triângulo (lados=" << lado1 << ", " << lado2 << ", " << lado3 << ")" << endl;
    cout << "  Área: " << calcular_area() << endl;
    cout << "  Perímetro: " << calcular_perimetro() << endl;
}
};

```

```
int main() {  
    cout << "=== Exemplo de Classes Abstratas - Formas ===" << endl << endl;  
  
    // Criando formas usando ponteiros para a classe base  
    vector<unique_ptr<Forma>> formas;  
  
    formas.push_back(make_unique<Circulo>(5.0));  
    formas.push_back(make_unique<Retangulo>(4.0, 6.0));  
    formas.push_back(make_unique<Triangulo>(3.0, 4.0, 5.0));  
  
    // Polimorfismo: cada forma se comporta de acordo com sua implementação  
    for (const auto& forma : formas) {  
        forma->exibir();  
        cout << endl;  
    }  
  
    return 0;  
}
```

INSTITUTO
TECNOLOGICO
EDUCACIONAL
DA AMAZÔNIA

Parado por Desenvolver Talentos

RECURSOS AVANÇADOS E PROJETO FINAL

AULA 11: LINGUAGEM DE PROGRAMAÇÃO C++

Templates

Código Genérico para Múltiplos Tipos

- Função template:

```
template <typename T>  
T somar(T a, T b) {  
    return a + b;  
}
```

- Classe template:

```
template <typename T>  
class Pilha { ... };
```


STL (Standard Template Library)

Containers e Algoritmos Prontos

- `vector`: array dinâmico (similar ao vetor, mas redimensionável)

```
#include <vector>
```

```
vector<int> numeros = {1, 2, 3, 4};
```

```
numeros.push_back(5);
```

- `string` (já utilizada)
- `sort`: ordenação
- `find`: busca

Manipulação de Arquivos (fstream)

Persistindo Dados em Disco

cpp

```
#include <fstream>
```

```
// Escrever
```

```
ofstream arquivo("dados.txt");
```

```
arquivo << "Texto a salvar\n";
```

```
arquivo.close();
```

```
// Ler
```

```
ifstream arquivo("dados.txt");
```

```
string linha;
```

```
while (getline(arquivo, linha)) {
```

```
    cout << linha << endl;
```

```
}
```

```
arquivo.close();
```

Considerações Finais

Próximos Passos e Encerramento

- C++ é a base para linguagens como Java, C# e JavaScript
- Conceitos aprendidos: ponteiros, POO, STL, arquivos
- Devolutiva final e dúvidas
- Parabéns pela conclusão da disciplina!